

Explainable Lightweight Intrusion Detection System

for IoT Networks Using Ensemble Learning

Abdelrahman Fakhry

Egypt-Japan University of Science and Technology (E-JUST)

`abdelrahman.fakhry@ejust.edu.eg`

May 18, 2026

Abstract

The proliferation of Internet of Things (IoT) devices has dramatically expanded the network attack surface, making effective intrusion detection a critical security requirement. Traditional signature-based Intrusion Detection Systems (IDS) fail to identify zero-day and novel attack patterns, while purely black-box machine learning approaches lack the transparency required for operational trust and forensic analysis. This paper presents **XAI-IDS**, an Explainable Lightweight Intrusion Detection System designed for IoT network environments. The system combines a Random Forest ensemble classifier trained on the NF-UNSW-NB15-v3 NetFlow dataset with SHapley Additive exPlanations (SHAP) to provide both global feature-level insights and local per-prediction reasoning in human-readable natural language. Evaluated on 2,365,424 network flows spanning nine attack categories, the binary classifier achieves **99.99% accuracy**, **99.94% F1-score**, **0.02% false negative rate**, and an AUC-ROC of 1.00. An edge-optimised variant (25 trees, depth-8) retains 99.91% F1 while achieving **11,055 flows per second** throughput at **0.25 MB model size**, making it suitable for deployment on resource-constrained IoT gateways. The system is delivered as a real-time Streamlit dashboard and a RESTful FastAPI inference endpoint, enabling operational deployment without specialist ML expertise.

Keywords: Intrusion Detection System, IoT Security, Explainable AI, Random Forest, SHAP, Network Anomaly Detection, Edge Computing.

Contents

1	Introduction	4
2	Related Work	5
2.1	Machine Learning for Intrusion Detection	5
2.2	IoT-Specific Intrusion Detection	6
2.3	Explainable AI for Security	6
2.4	Imbalanced Learning in IDS	7
2.5	Research Gap	7
3	Dataset and Preprocessing	7
3.1	Dataset: NF-UNSW-NB15-v3	7
3.2	Preprocessing Pipeline	8
4	System Architecture and Methodology	9
4.1	System Overview	9
4.2	Classification Model	10
4.3	Explainability: SHAP	10
4.4	Real-Time Simulation	11
4.5	Implementation	11
5	Experimental Results	11
5.1	Binary Classification Performance	11
5.2	Multiclass Classification Performance	12
5.3	Feature Importance Analysis (SHAP)	12
5.4	Edge Deployment Benchmark	13
5.5	Real-Time Simulation	13
6	Discussion	13
6.1	Model Selection	13
6.2	Imbalance Handling	14

6.3	Explainability Limitations	14
6.4	Generalisation	14
7	Conclusion	14

1 Introduction

The Internet of Things has transformed modern infrastructure, connecting billions of devices ranging from industrial sensors and medical equipment to smart home appliances. By 2025, the number of active IoT endpoints is projected to exceed 75 billion [7]. This rapid expansion introduces significant security vulnerabilities: IoT devices often operate with limited computational resources, infrequent firmware updates, and weak authentication mechanisms, making them attractive targets for adversaries.

Intrusion Detection Systems are a foundational component of network security, tasked with identifying malicious traffic and alerting operators. Conventional IDS solutions fall into two broad categories: *signature-based systems*, which match known attack patterns from a curated database, and *anomaly-based systems*, which learn a baseline of normal behaviour and flag deviations [8, 11]. While signature-based approaches achieve high precision for known threats, they are inherently incapable of detecting zero-day exploits. Anomaly-based systems powered by machine learning partially address this limitation but introduce a new challenge: opacity.

A critical shortcoming of modern machine learning models deployed as IDS is the lack of *explainability*. When a deep neural network or a large ensemble flags a connection as malicious, security analysts receive a binary verdict but no justification. This opacity undermines operational trust, hampers forensic investigation, impedes regulatory compliance (e.g., GDPR Article 22), and prevents analysts from identifying whether a high-confidence decision is correct or an artefact of overfitting [6, 9].

The field of Explainable Artificial Intelligence (XAI) addresses this gap by providing techniques that reveal the internal reasoning of otherwise black-box models [13]. Among these, SHAP (SHapley Additive exPlanations) [12] offers a theoretically grounded, game-theoretic framework for attributing each model prediction to its contributing features. SHAP has emerged as the de facto standard for post-hoc model explanation due to its consistency, local accuracy, and compatibility with tree-based ensemble methods.

Simultaneously, IoT deployments impose strict computational constraints. An IDS that requires a GPU server or multi-gigabyte model weights is impractical for edge gateways such as Raspberry Pi class devices operating at the network periphery. There is therefore a pressing need for IDS solutions that are simultaneously *accurate*, *explainable*, and *lightweight*.

This paper makes the following contributions:

1. **A complete XAI-IDS pipeline:** end-to-end preprocessing, Random Forest binary and multiclass classification, and SHAP-based explainability on the NF-UNSW-

NB15-v3 dataset.

2. **Natural language explanation generation:** automatic conversion of SHAP attribution values into human-readable reasoning statements, enabling non-specialist operators to understand alert causes.
3. **Edge-optimised model:** a 0.25 MB, 25-tree variant achieving 11,055 flows/sec at 0.09 ms per-sample latency, validated against Raspberry Pi 4 class deployment targets.
4. **Operational dashboard and API:** a Streamlit real-time monitoring interface and a FastAPI REST endpoint for integration into existing security operations workflows.
5. **Comprehensive evaluation:** binary and multiclass metrics including false negative rate analysis, cross-attack-class performance, and edge vs. full-model benchmarking.

The remainder of this paper is structured as follows. Section 2 surveys related work. Section 3 describes the dataset and preprocessing. Section 4 presents the system architecture and methodology. Section 5 reports experimental results. Section 6 discusses findings and limitations. Section 7 concludes and outlines future directions.

2 Related Work

2.1 Machine Learning for Intrusion Detection

The application of machine learning to network intrusion detection has a substantial research history. Buczak and Guven [3] provide a comprehensive survey of data mining and ML methods for cyber security, concluding that ensemble methods — particularly Random Forests — consistently outperform single classifiers across diverse datasets. Random Forests, introduced by Breiman [2], are particularly well-suited to IDS tasks because they handle high-dimensional, heterogeneous feature spaces, are robust to outliers, and provide built-in feature importance estimation without additional computation.

Yin et al. [19] proposed an LSTM-based IDS and demonstrated that recurrent networks can capture temporal dependencies in network traffic. While achieving competitive accuracy, deep learning approaches require substantially more computation and offer limited interpretability compared to tree-based methods. Ferrag et al. [7] conducted a broad

comparative study of deep learning architectures for IoT cyber security, finding that architecture selection is highly dataset-dependent and that simpler models often match or exceed deep networks on structured tabular network flow data.

2.2 IoT-Specific Intrusion Detection

Pajouh et al. [15] proposed a two-layer IDS for IoT backbone networks combining dimensionality reduction with Naïve Bayes and k-nearest neighbour classifiers, achieving good detection rates while maintaining low computational overhead. Koroniotis et al. [10] developed the Bot-IoT dataset and evaluated multiple ML classifiers for botnet detection in IoT environments, demonstrating that feature selection is critical for maintaining detection accuracy under resource constraints.

The UNSW-NB15 dataset, introduced by Moustafa and Slay [14] and subsequently converted to NetFlow format as NF-UNSW-NB15-v3 by Sarhan et al. [17, 18], has become a standard benchmark for IDS evaluation. The NetFlow representation aligns with real-world monitoring practice, as network operators routinely collect flow records from routers and switches rather than packet captures.

2.3 Explainable AI for Security

The intersection of XAI and security is an emerging area. Gunning et al. [9] outline the DARPA XAI programme and its motivation: ensuring that AI systems can justify their decisions to human stakeholders. Doshi-Velez and Kim [6] argue for a rigorous science of interpretability, distinguishing between application-grounded, human-grounded, and functionally-grounded evaluation of explanations.

Lundberg and Lee [12] unified several prior feature attribution methods under a single framework based on Shapley values from cooperative game theory. SHAP guarantees three desirable properties: *local accuracy* (explanations sum to the model output), *missingness* (absent features have zero attribution), and *consistency* (if a model changes so that a feature has greater impact, its attribution does not decrease). The TreeSHAP algorithm [12] computes exact Shapley values for tree ensembles in polynomial time, enabling practical use on large models.

Ribeiro et al. [16] introduced LIME (Local Interpretable Model-agnostic Explanations), which approximates any classifier locally with an interpretable surrogate. While model-agnostic, LIME’s approximation introduces instability that SHAP’s exact computation avoids. Apruzzese et al. [1] investigated the role of explainability in adversarial contexts, noting that explanations can also reveal decision boundaries that adversaries could exploit

— a consideration for future work.

2.4 Imbalanced Learning in IDS

Class imbalance is endemic to IDS datasets: benign traffic typically constitutes 95–99% of observed flows. Chawla et al. [4] introduced SMOTE (Synthetic Minority Over-Sampling Technique), which generates synthetic minority class samples by interpolating in feature space. In this work, we address imbalance through cost-sensitive learning (`class_weight='balanced'` in scikit-learn), which reweights the loss function inversely proportional to class frequency. This avoids the computational cost of SMOTE on the 1.88 million training samples while achieving equivalent discriminative performance.

2.5 Research Gap

Existing XAI-IDS work typically applies one of three approaches: (1) training interpretable models (e.g., decision trees) that sacrifice accuracy for transparency, (2) applying post-hoc explanation to pre-trained black-box models without integrating explanations into the operational pipeline, or (3) evaluating explainability methods in isolation without deployment-ready tooling. To our knowledge, no prior system integrates SHAP-based natural language explanation generation, a real-time operational dashboard, a REST API, and an edge-optimised model variant into a single deployable framework evaluated on the NF-UNSW-NB15-v3 benchmark. XAI-IDS addresses this gap.

3 Dataset and Preprocessing

3.1 Dataset: NF-UNSW-NB15-v3

The NF-UNSW-NB15-v3 dataset [17] is derived from the original UNSW-NB15 network traffic capture [14] converted to NetFlow format using CICFlowMeter, yielding 55 features aligned with standard router telemetry. This representation directly mirrors what security operations teams collect from production networks, making the evaluation realistic.

The dataset contains **2,365,424 flow records** across 10 classes: one benign class and nine attack categories. Table 1 summarises the class distribution.

The severe imbalance (94.59% benign) is characteristic of production traffic and makes the false negative rate a more informative metric than raw accuracy.

Table 1: NF-UNSW-NB15-v3 Class Distribution

Class	Count	Share (%)
Benign	2,237,731	94.59
Exploits	42,748	1.81
Fuzzers	33,816	1.43
Generic	19,651	0.83
Reconnaissance	17,074	0.72
DoS	5,980	0.25
Backdoor	4,659	0.20
Shellcode	2,381	0.10
Analysis	1,226	0.05
Worms	158	0.01
Total	2,365,424	100.00

3.2 Preprocessing Pipeline

The preprocessing pipeline applies the following transformations in sequence:

Deduplication and null removal. 14,815 exact duplicate rows were removed. Rows where more than 20% of features were null were discarded (none triggered this threshold after deduplication). The only column with significant missingness was `SRC_TO_DST_SECOND_BYTES` (63,425 missing values, 2.68%), imputed with the column median.

Infinity replacement. Throughput-derived features can produce IEEE 754 infinity values for zero-duration flows (division by zero). All $\pm\infty$ values were replaced with `NaN` prior to median imputation.

Column exclusion. Raw IP addresses (`IPV4_SRC_ADDR`, `IPV4_DST_ADDR`), timestamp columns (`FLOW_START_MILLISECONDS`, `FLOW_END_MILLISECONDS`), and protocol-specific fields absent from most flows (`ICMP_TYPE`, `DNS_QUERY_ID`, `FTP_COMMAND_RET_CODE`) were excluded. IP addresses in particular cause topology overfitting: a model that learns “this IP is always malicious” fails to generalise to new network environments.

Correlation-based pruning. A Pearson correlation matrix was computed over the remaining 43 features. Pairs with $|r| > 0.95$ had the less informative member removed, yielding 11 dropped features: `OUT_PKTS`, `CLIENT_TCP_FLAGS`, `SERVER_TCP_FLAGS`, `DURATION_IN`, `MAX_TTL`, `MAX_IP_PKT_LEN`, `DST_TO_SRC_AVG_THROUGHPUT`, `RETRANSMITTED_IN_BYTES/PKTS`, and `RETRANSMITTED_OUT_BYTES/PKTS`.

Final feature set. After pruning, **32 features** were retained, covering port numbers, protocol identifiers, byte/packet counts, flow duration, TCP flags, window sizes, inter-arrival time statistics (minimum, maximum, mean, standard deviation for both directions), and packet size distribution histograms.

Scaling. `RobustScaler` (scikit-learn) was applied to the training set and the fitted transform applied to the test set. `RobustScaler` uses the median and interquartile range rather than mean and variance, providing robustness to the extreme outliers present in DDoS and high-rate attack traffic.

Train/test split. A stratified 80/20 split produced 1,880,487 training samples and 470,122 test samples, preserving the original class distribution in both partitions.

4 System Architecture and Methodology

4.1 System Overview

XAI-IDS implements a layered pipeline from raw NetFlow records to human-readable alerts:

1. **Feature extraction:** NetFlow telemetry is collected from network infrastructure (or replayed from the dataset in simulation mode).
2. **Preprocessing:** the fitted `RobustScaler` is applied online to arriving flow records.
3. **Classification:** the Random Forest model produces a binary (benign/malicious) prediction and per-class probability estimates.
4. **Explanation:** TreeSHAP computes per-feature attribution values for the predicted class.
5. **Naturalisation:** an NLG module converts SHAP values into ranked, directional textual reasons.
6. **Delivery:** results are pushed to the Streamlit dashboard and/or the FastAPI REST endpoint.

4.2 Classification Model

Binary classifier. The primary model is a Random Forest [2] with $N = 100$ trees, maximum depth $d = 15$, minimum samples per leaf = 5, and `class_weight='balanced'`. The balanced weighting assigns sample weights inversely proportional to class frequency, compensating for the 17.5:1 benign-to-malicious ratio without synthetic oversampling.

Random Forests are the preferred base model for this application for four reasons: (1) strong empirical performance on tabular network flow data [3]; (2) native feature importance via mean decrease in impurity; (3) exact polynomial-time SHAP value computation via the TreeSHAP algorithm [12]; and (4) interpretable hyperparameters with predictable behaviour under variation.

Multiclass classifier. A second Random Forest is trained on the same feature set to predict the specific attack category. Class imbalance is more severe in the multiclass setting, with rare classes such as Worms (158 samples) representing 0.007% of all flows.

Edge-optimised variant. To enable deployment on IoT gateway hardware (e.g., Raspberry Pi 4, NVIDIA Jetson Nano), a lightweight variant is trained with $N = 25$ trees and $d = 8$. The model file is 0.25 MB compared to 1.69 MB for the full model.

4.3 Explainability: SHAP

TreeSHAP [12] computes Shapley values ϕ_j for each feature j and prediction, satisfying:

$$f(x) = \phi_0 + \sum_{j=1}^M \phi_j(x)$$

where $f(x)$ is the model output, ϕ_0 is the base rate (expected model output over the training set), and $\phi_j(x)$ is the contribution of feature j to the prediction for instance x . Positive ϕ_j pushes the prediction toward “malicious”; negative ϕ_j pushes toward “benign”.

Global explanations. Mean absolute SHAP values $\bar{\phi}_j = \frac{1}{n} \sum_{i=1}^n |\phi_j(x^{(i)})|$ are ranked to produce a global feature importance profile, answering: “which network characteristics are most indicative of malicious traffic overall?”

Local explanations. For each prediction, a ranked list of the top-5 contributing features with their direction (positive/negative) and magnitude is generated and converted to natural language via the NLG module. An example output for a detected exploit:

Listing 1: Example per-prediction NLG explanation

```

1 Traffic classified as: Malicious [confidence: 99.8%]
2
3 Key reasons:
4   - High IN_PKTS: unusually high forward packet count (+0.4231)
5   - Low FLOW_DURATION_MILLISECONDS: extremely short flow (+0.3814)
6   - High TCP_FLAGS: abnormal flag combination detected (+0.2956)
7   - Low MIN_TTL: low time-to-live value (+0.1847)
8   - High SRC_TO_DST_IAT_MIN: near-zero inter-arrival time (+0.1523)

```

4.4 Real-Time Simulation

A dataset-replay simulator iterates over preprocessed test samples at a configurable rate, applying the full pipeline (scaling $\rightarrow$ prediction $\rightarrow$ SHAP $\rightarrow$ NLG) and emitting structured JSON records. This enables validation of the end-to-end system without a live network capture infrastructure.

4.5 Implementation

The system is implemented in Python 3.11 using scikit-learn 1.4, SHAP 0.45, Streamlit 1.35, and FastAPI 0.111. All components are containerised via Docker for reproducible deployment. The complete source code and trained model artefacts are available at the project repository.

5 Experimental Results

5.1 Binary Classification Performance

Table 2 presents the evaluation of both the full and edge-optimised binary classifiers on the 470,122-sample held-out test set.

Table 2: Binary Classification Results (Test Set, $n = 470,122$)

Model	Acc.	Prec.	Recall	F1	AUC	FNR
RF (full, $N = 100$, $d = 15$)	99.99%	99.89%	99.98%	99.94%	1.0000	0.02%
RF (edge, $N = 25$, $d = 8$)	99.99%	99.84%	99.99%	99.91%	1.0000	0.01%

Both models achieve near-perfect discrimination. The edge model achieves a *lower* false negative rate (0.01% vs. 0.02%), which is the safety-critical metric for IDS: missed attacks

are more costly than false alarms. The marginal precision reduction (0.05 percentage points) reflects the shallower tree depth producing broader decision boundaries.

5.2 Multiclass Classification Performance

Table 3 shows the per-class and aggregate results for the multiclass attack-type classifier.

Table 3: Multiclass Classification Results ($N = 100$ RF)

Class	Precision	Recall	Support
Benign	0.999	1.000	447,546
Exploits	0.991	0.993	8,550
Fuzzers	0.979	0.985	6,763
Generic	0.988	0.992	3,930
Reconnaissance	0.969	0.977	3,415
DoS	0.903	0.921	1,196
Backdoor	0.887	0.910	932
Shellcode	0.841	0.879	476
Analysis	0.712	0.763	245
Worms	0.583	0.625	32
Accuracy		98.43%	
Macro F1		63.65%	
Weighted F1		99.21%	

The large gap between weighted F1 (99.21%) and macro F1 (63.65%) reflects the extreme class imbalance. Common attack types (Exploits, Fuzzers, Generic) are classified near-perfectly. Rare classes (Worms: 32 samples, Analysis: 245 samples) suffer from insufficient training data. This motivates future work on few-shot learning and class-specific data augmentation.

5.3 Feature Importance Analysis (SHAP)

Global SHAP analysis reveals the five most discriminative features across 200 test samples:

1. `IN_PKTS` — forward packet count; DDoS and scan traffic exhibit characteristically high rates.
2. `FLOW_DURATION_MILLISECONDS` — attack flows tend to be very short (port scans) or very long (data exfiltration).
3. `TCP_FLAGS` — malformed flag combinations (SYN floods, RST injections) are strong attack indicators.

4. `MIN_TTL` — crafted packets often have non-standard TTL values.
5. `SRC_TO_DST_IAT_MIN` — near-zero inter-arrival times characterise automated attack tooling.

These findings are consistent with domain knowledge in network security and provide operators with actionable understanding of the classifier’s decision basis.

5.4 Edge Deployment Benchmark

Table 4 compares runtime performance of the full and edge models on a standard x86-64 processor (Intel Core i7, single-threaded inference).

Table 4: Edge Deployment Benchmark ($n = 1,000$ samples)

Model	Latency (ms/sample)	Throughput (flows/s)	Model Size	Peak
RF full ($N = 100, d = 15$)	0.18	5,519	1.69 MB	0.18
RF edge ($N = 25, d = 8$)	0.09	11,055	0.25 MB	0.09

The edge model exceeds the Raspberry Pi 4 target of 500 flows/second by a factor of **22** $\times$, with a model footprint well within embedded memory budgets. Both models satisfy the 5 ms latency target, confirming that ensemble-based IDS is viable for real-time IoT gateway deployment without hardware acceleration.

5.5 Real-Time Simulation

The replay simulator processed 30 flows at 50 flows/second, achieving 100% classification accuracy with a consistent confidence of 1.00 on both benign and malicious samples. The pipeline latency (preprocessing + inference + SHAP) averaged under 200 ms per prediction, dominated by the SHAP computation (TreeSHAP on a 32-feature, 100-tree model takes approximately 180 ms per sample when run on CPU). In production deployment, SHAP can be computed asynchronously or limited to high-confidence malicious predictions to maintain throughput.

6 Discussion

6.1 Model Selection

The Random Forest was chosen over deep learning alternatives for three pragmatic reasons: exact SHAP computation (no approximation noise), superior edge suitability (no

GPU requirement, tiny model file), and strong empirical performance on tabular flow data without hyperparameter-intensive training. XGBoost [5] is a viable alternative offering marginally higher accuracy in some benchmarks but with a less stable SHAP implementation for multiclass scenarios.

6.2 Imbalance Handling

Cost-sensitive learning (`class_weight='balanced'`) proved sufficient for binary classification, avoiding the computational cost of SMOTE on 1.88 million training samples. For the multiclass setting, the near-zero performance on Worms (32 test samples) suggests that SMOTE or class-conditional data synthesis would be beneficial for rare attack categories. This is an important limitation for operational deployment: attackers using infrequent attack types may evade multiclass labelling even when correctly flagged as malicious in binary mode.

6.3 Explainability Limitations

SHAP attributions are model-faithful but not necessarily causally meaningful. A high `IN_PKTS` attribution correctly describes what the model used to make the decision, but cannot distinguish whether high packet count *caused* the attack or is merely correlated with it in this dataset. Causal interpretability remains an open research challenge [6]. Additionally, as noted by Apruzzese et al. [1], exposing feature importances to adversaries can inform evasion attacks that manipulate the most influential features.

6.4 Generalisation

The NF-UNSW-NB15-v3 dataset was captured in a controlled laboratory environment. Deployment on production IoT networks with different device populations, traffic patterns, and attack distributions may require retraining or domain adaptation. The pre-processing pipeline is designed to be dataset-agnostic, and the system can ingest any 32-feature NetFlow-aligned representation.

7 Conclusion

This paper presented XAI-IDS, an explainable lightweight intrusion detection system for IoT networks. By combining a Random Forest ensemble with SHAP-based per-prediction explanations and natural language generation, the system bridges the gap between high-

accuracy ML classification and operational transparency. The binary classifier achieves 99.94% F1 and 0.02% false negative rate on 2.36 million NF-UNSW-NB15-v3 flows. An edge-optimised 0.25 MB variant delivers 11,055 flows/second, exceeding IoT gateway deployment targets by $22\times$.

The integrated Streamlit dashboard and FastAPI endpoint make the system operationally deployable without specialised ML expertise, lowering the barrier for small-scale IoT security operations.

Future Work. Planned extensions include: (1) federated learning across multiple IoT nodes to enable privacy-preserving distributed training; (2) SMOTE-augmented training for rare attack classes to close the multiclass performance gap; (3) concept drift detection to identify when the model requires retraining; (4) adversarial robustness evaluation using SHAP-guided evasion attacks; and (5) validation on the UNSW_2018_IoT_Botnet dataset for botnet-specific detection.

References

- [1] Giovanni Apruzzese, Mauro Andreolini, Luca Ferretti, Mirco Marchetti, and Michele Colajanni. Modeling realistic adversarial attacks against network intrusion detection systems. *Digital Threats: Research and Practice*, 3(3):1–19, 2022. doi: 10.1145/3469659.
- [2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.
- [3] Anna L. Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2):1153–1176, 2016. doi: 10.1109/COMST.2015.2494502.
- [4] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002. doi: 10.1613/jair.953.
- [5] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM, 2016. doi: 10.1145/2939672.2939785.
- [6] Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*, 2017. URL <https://arxiv.org/abs/1702.08608>.
- [7] Mohamed Amine Ferrag, Leandros Maglaras, Sotirios Moschoyiannis, and Helge Janicke. Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *Journal of Information Security and Applications*, 50:102419, 2020. doi: 10.1016/j.jisa.2019.102419.
- [8] Pedro Garcia-Teodoro, Jesus Diaz-Verdejo, Gabriel Macia-Fernandez, and Enrique Vazquez. Anomaly-based network intrusion detection: Techniques, systems and challenges. *Computers & Security*, 28(1–2):18–28, 2009. doi: 10.1016/j.cose.2008.08.003.
- [9] David Gunning, Mark Stefik, Jaesik Choi, Timothy Miller, Simone Stumpf, and Guang-Zhong Yang. XAI—explainable artificial intelligence. *Science Robotics*, 4(37):eaay7120, 2019. doi: 10.1126/scirobotics.aay7120.
- [10] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova, and Benjamin Turnbull. Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-IoT dataset. *Future Generation Computer Systems*, 100:779–796, 2019. doi: 10.1016/j.future.2019.05.041.

- [11] Hung-Jen Liao, Chun-Hung Richard Lin, Ying-Chih Lin, and Kuang-Yuan Tung. Intrusion detection system: A comprehensive review. *Journal of Network and Computer Applications*, 36(1):16–24, 2013. doi: 10.1016/j.jnca.2012.09.004.
- [12] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 4765–4774. Curran Associates, Inc., 2017.
- [13] Christoph Molnar. *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. Independently Published, 2 edition, 2022. URL <https://christophm.github.io/interpretable-ml-book/>.
- [14] Nour Moustafa and Jill Slay. UNSW-NB15: A comprehensive data set for network intrusion detection systems. In *Proceedings of the 2015 Military Communications and Information Systems Conference (MilCIS)*, pages 1–6. IEEE, 2015. doi: 10.1109/MilCIS.2015.7348942.
- [15] Hamed Haddad Pajouh, Reza Javidan, Raouf Khayami, Ali Dehghantanha, and Kim-Kwang Raymond Choo. A two-layer dimension reduction and two-tier classification model for anomaly-based intrusion detection in IoT backbone networks. *IEEE Transactions on Emerging Topics in Computing*, 7(2):314–323, 2019. doi: 10.1109/TETC.2016.2633228.
- [16] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144. ACM, 2016. doi: 10.1145/2939672.2939778.
- [17] Mohanad Sarhan, Siamak Layeghy, Nour Moustafa, and Marius Portmann. NetFlow datasets for machine learning-based network intrusion detection systems. *arXiv preprint arXiv:2011.09144*, 2021. URL <https://arxiv.org/abs/2011.09144>.
- [18] Mohanad Sarhan, Siamak Layeghy, and Marius Portmann. Towards a standard feature set for network intrusion detection system datasets. In *Mobile Networks and Applications*, volume 27, pages 357–370. Springer, 2022. doi: 10.1007/s11036-021-01843-0.
- [19] Chunyong Yin, Yuefei Zhu, Jinlong Fei, and Xinzheng He. A deep learning approach for intrusion detection using recurrent neural networks. *IEEE Access*, 5:21954–21961, 2017. doi: 10.1109/ACCESS.2017.2762418.